

Министерство Образования Республики Беларусь
Учреждение Образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 3
По дисциплине «Современные платформы программирования»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
06.03.2026

Цель работы: Приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Ход работы:

Первая группа заданий (порождающий паттерн)

10) Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.

Выбранный паттерн: Абстрактная фабрика (Abstract Factory)

```
class Sedan:
    def drive(self):
        pass
```

```
class Hatchback:
    def drive(self):
        pass
```

```
class ToyotaSedan(Sedan):
    def drive(self):
        return "Toyota Camry"
```

```
class ToyotaHatchback(Hatchback):
    def drive(self):
        return "Toyota Corolla"
```

```
class TeslaSedan(Sedan):
    def drive(self):
        return "Tesla Model 3"
```

```
class TeslaHatchback(Hatchback):
    def drive(self):
        return "Tesla Model Y"
```

```
class CarFactory:
    def create_sedan(self):
        pass
```

```

def create_hatchback(self):
    pass

class ToyotaFactory(CarFactory):
    def create_sedan(self):
        return ToyotaSedan()

    def create_hatchback(self):
        return ToyotaHatchback()

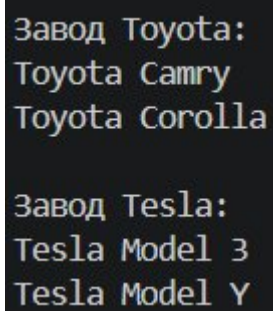
class TeslaFactory(CarFactory):
    def create_sedan(self):
        return TeslaSedan()

    def create_hatchback(self):
        return TeslaHatchback()

def produce_cars(factory, name):
    print(f"\nЗавод {name}:")
    print(factory.create_sedan().drive())
    print(factory.create_hatchback().drive())

produce_cars(ToyotaFactory(), "Toyota")
produce_cars(TeslaFactory(), "Tesla")

```



```

Завод Toyota:
Toyota Camry
Toyota Corolla

Завод Tesla:
Tesla Model 3
Tesla Model Y

```

Вторая группа заданий (структурный паттерн)

10) Учетная запись покупателя книжного интернет-магазина. Предусмотреть различные уровни учетки в зависимости от активности покупателя.

Дополнительные уровни добавляют функциональные возможности и открывают доступ к уникальным предложениям.
Выбранный паттерн: Декоратор (Decorator)

```
class Account:
    def get_discount(self):
        return 0

    def get_benefits(self):
        return []

    def get_name(self):
        return "Базовая"

class BaseAccount(Account):
    def get_discount(self):
        return 0

    def get_benefits(self):
        return ["Просмотр книг"]

    def get_name(self):
        return "Базовая"

class SilverLevel(Account):
    def __init__(self, account):
        self.account = account

    def get_discount(self):
        return self.account.get_discount() + 5

    def get_benefits(self):
        benefits = self.account.get_benefits()
        benefits.append("Скидка 5%")
        benefits.append("Бесплатная доставка")
        return benefits

    def get_name(self):
        return f"{self.account.get_name()} + Серебряный"
```

```

class GoldLevel(Account):
    def __init__(self, account):
        self.account = account

    def get_discount(self):
        return self.account.get_discount() + 10

    def get_benefits(self):
        benefits = self.account.get_benefits()
        benefits.append("Скидка 10%")
        benefits.append("Бесплатная доставка за 1 час")
        benefits.append("Доступ к эксклюзивным книгам")
        benefits.append("Приоритетная поддержка")
        return benefits

    def get_name(self):
        return f'{self.account.get_name()} + Золотой'

def show_account_info(account):
    print(f"\n=== {account.get_name()} ===")
    print(f"Скидка: {account.get_discount()}%")
    print("Возможности:")
    for benefit in account.get_benefits():
        print(f" - {benefit}")

base = BaseAccount()
show_account_info(base)
silver = SilverLevel(base)
show_account_info(silver)
gold = GoldLevel(silver)
show_account_info(gold)
print("\n=== Покупатель заказал книгу ===")
book_price = 1000
final_price = book_price - (book_price * gold.get_discount() / 100)
print(f"Цена книги: {book_price} руб.")
print(f"Скидка: {gold.get_discount()}%")
print(f"Итого: {final_price} руб.")

```

```

=== Базовая ===
Скидка: 0%
Возможности:
- Просмотр книг

=== Базовая + Серебряный ===
Скидка: 5%
Возможности:
- Просмотр книг
- Скидка 5%
- Бесплатная доставка

=== Базовая + Серебряный + Золотой ===
Скидка: 15%
Возможности:
- Просмотр книг
- Скидка 5%
- Бесплатная доставка
- Скидка 10%
- Бесплатная доставка за 1 час
- Доступ к эксклюзивным книгам
- Приоритетная поддержка

=== Покупатель заказал книгу ===
Цена книги: 1000 руб.
Скидка: 15%
Итого: 850.0 руб.

```

Третья группа заданий (поведенческий паттерн)

10) Проект «Банкомат». Предусмотреть выполнение основных операций (ввод пин-кода, снятие суммы, завершение работы) и наличие различных режимов работы (ожидание, аутентификация, выполнение операции, блокировка – если нет денег). Атрибуты: общая сумма денег в банкомате, ID.

Выбранный паттерн: Состояние (State)

```

class State:
    def __init__(self, atm):
        self.atm = atm

class WaitState(State):
    def insert_card(self):
        print("Карта вставлена. Введите PIN")
        self.atm.state = self.atm.auth

    def enter_pin(self, pin):
        print("Сначала вставьте карту")

    def withdraw(self, amount):
        print("Сначала вставьте карту")

    def end(self):

```

```
print("Нет сессии")
```

```
class AuthState(State):  
    def insert_card(self):  
        print("Карта уже есть")  
  
    def enter_pin(self, pin):  
        if pin == "1234":  
            print("PIN верен")  
            self.atm.state = self.atm.oper  
        else:  
            print("PIN неверен")  
            self.atm.state = self.atm.wait
```

```
    def withdraw(self, amount):  
        print("Сначала введите PIN")
```

```
    def end(self):  
        print("Сессия завершена")  
        self.atm.state = self.atm.wait
```

```
class OperState(State):  
    def insert_card(self):  
        print("Карта уже есть")
```

```
    def enter_pin(self, pin):  
        print("PIN уже введен")
```

```
    def withdraw(self, amount):  
        if amount > self.atm.cash:  
            print("Нет денег в банкомате")  
        elif amount > 10000:  
            print("Лимит 10000 руб")  
        else:  
            self.atm.cash -= amount  
            print(f"Выдано {amount} руб. Остаток: {self.atm.cash}")
```

```
    print("Заберите карту")  
    self.atm.state = self.atm.wait
```

```
    def end(self):  
        print("Заберите карту")  
        self.atm.state = self.atm.wait
```

```
class BlockState(State):
    def insert_card(self):
        print("Банкомат заблокирован")
```

```
    def enter_pin(self, pin):
        print("Банкомат заблокирован")
```

```
    def withdraw(self, amount):
        print("Банкомат заблокирован")
```

```
    def end(self):
        print("Банкомат заблокирован")
```

```
class ATM:
    def __init__(self, cash):
        self.cash = cash
        self.wait = WaitState(self)
        self.auth = AuthState(self)
        self.oper = OperState(self)
        self.block = BlockState(self)
        self.state = self.wait
```

```
    def insert_card(self):
        self.state.insert_card()
```

```
    def enter_pin(self, pin):
        self.state.enter_pin(pin)
```

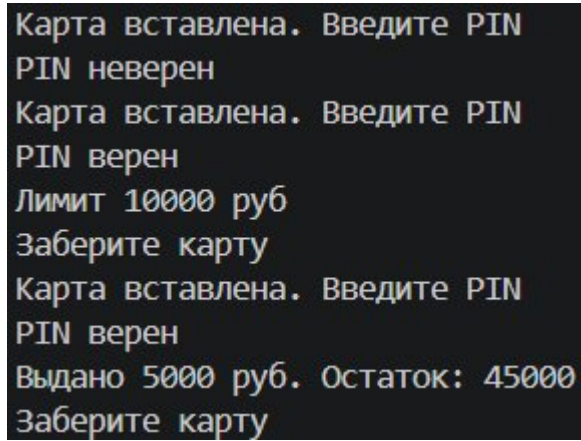
```
    def withdraw(self, amount):
        self.state.withdraw(amount)
```

```
    def end(self):
        self.state.end()
```

```
atm = ATM(50000)
atm.insert_card()
atm.enter_pin("1111")
atm.insert_card()
atm.enter_pin("1234")
atm.withdraw(15000)
```



```
atm.insert_card()  
atm.enter_pin("1234")  
atm.withdraw(5000)
```



```
Карта вставлена. Введите PIN  
PIN неверен  
Карта вставлена. Введите PIN  
PIN верен  
Лимит 10000 руб  
Заберите карту  
Карта вставлена. Введите PIN  
PIN верен  
Выдано 5000 руб. Остаток: 45000  
Заберите карту
```

Вывод работы: Приобрели навыки применения паттернов проектирования при решении практических задач с использованием языка Python.